Automated Detection of Android Lifecycle Anti-Patterns: From Static Analysis to Fine Tuned Large Language Models

PhD Dissertation Proposal Defense

May 11, 2026



Presenter: Muhammad Tehmasib Ali Tashfeen
Advisor: Dr. Zhiyong Shan
Software Analysis and Intelligence Lab - JB 256

School of Computing
Wichita State University

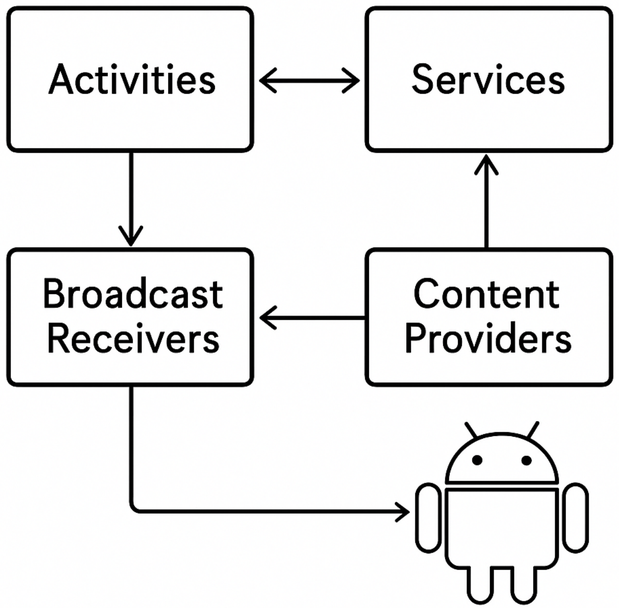
Committee:

- **Dr. Rajiv Bagai**
- **Dr. Abu Asaduzzaman**
- **Dr. Sergio A. Salinas Monroy**
- **Dr. Gisuk Hwang**

Intro: The Core Problem?

Android Components?

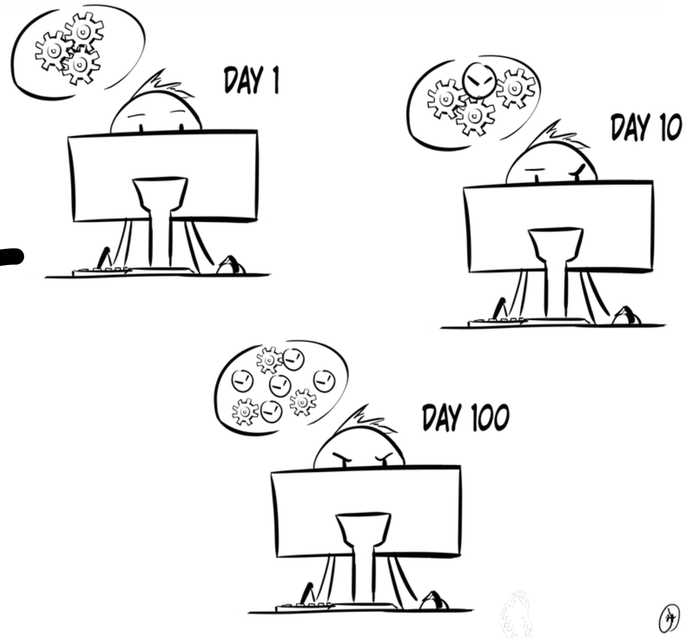
Component	Meaning
Activity	A screen in the app
Fragment	A reusable part of a screen
View	Buttons, text fields, layouts, UI objects
ViewModel	Holds data for UI
Service	Runs background work
Thread / Callback	Handles background tasks or events



Android lifecycle?

What is an anti-pattern?





The Core Problem?

Android lifecycle anti-patterns?

Why they happen?

Lifecycle Mismatches Cause Memory Leaks

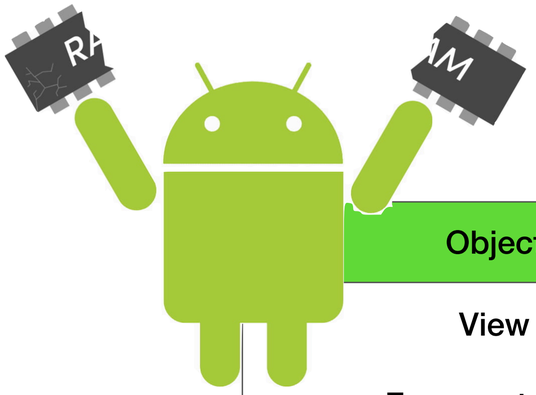
Android Framework | Application logic

```
private FragmentExampleBinding binding;

@Override
public View onCreateView(...) {
    binding = FragmentExampleBinding.inflate(inflater, container, false);
    return binding.getRoot();
}
```

onCreate()
onStart()
onResume()
onPause()
onStop()
onDestroy()





Object	Typical lifetime
View	Short-lived
Fragment view	Short-lived
Activity	Medium-lived
Fragment	Can live longer than its view

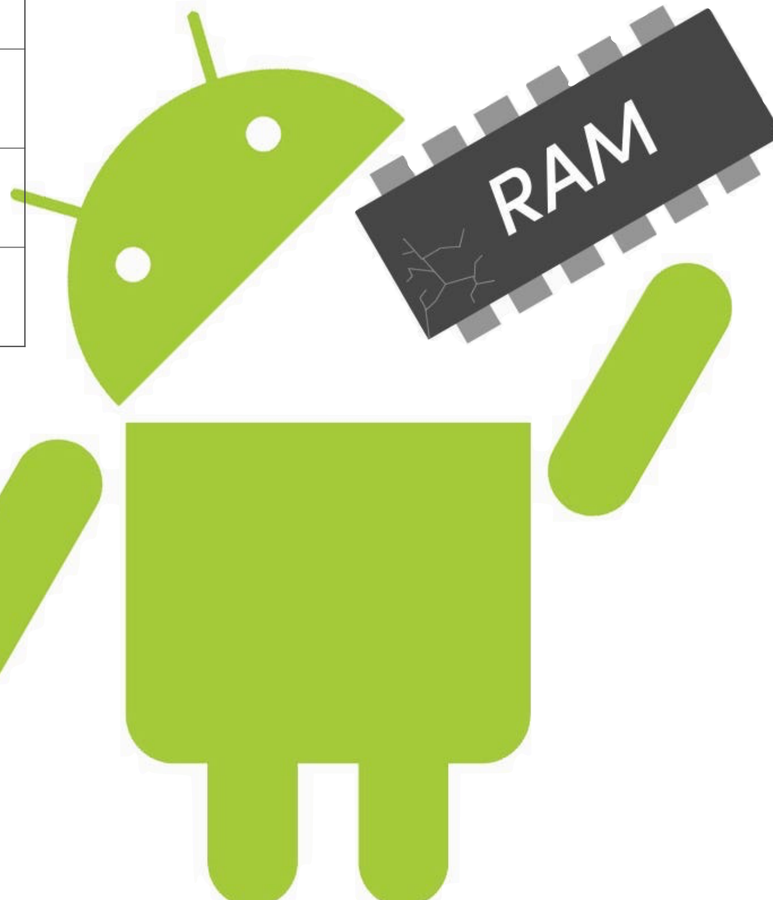
Significance: The Core Problem?

Risks

Risk	Meaning
Memory leak risk	Object stays in memory when it should be removed
Resource leak risk	Service, callback, or connection is not released
UI retention risk	Activity, Fragment, or View is kept by background work
Lifecycle mismatch risk	Object lifetime does not match Android lifecycle

Results





Why Existing Tools Fall Short?

Android Lint

Framework-aware, but **not organized** around a unified lifecycle-retention taxonomy. Covers only selected patterns.

LeakCanary

Confirms runtime leaks dynamically, but **not suitable for corpus-scale static screening**.

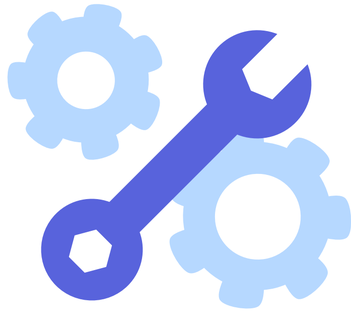
SpotBugs

Mature Java bytecode analysis, but **lacks Android lifecycle semantics**. Warnings are intentionally general.

FlowDroid

Soot-based APK analysis focused on **taint/information flow**, not lifecycle-retention patterns.

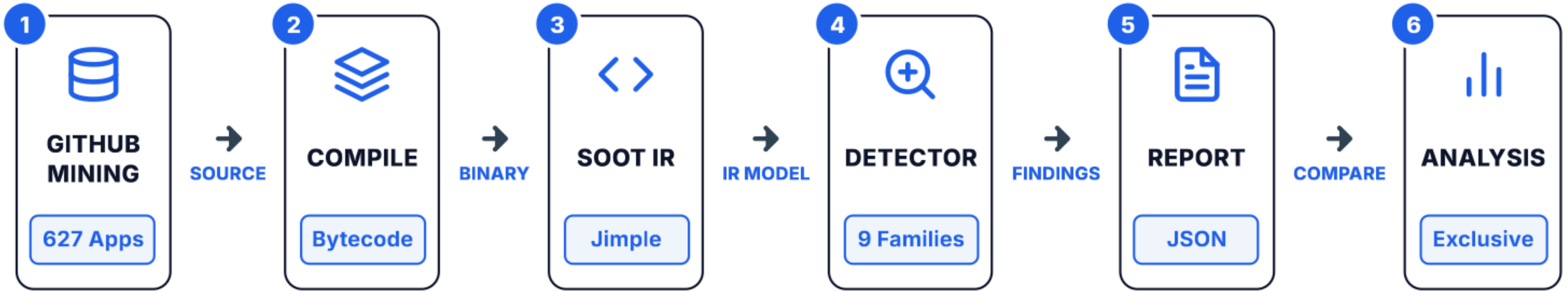
⚠ No existing tool provides a unified, lifecycle-aware static detector suite for Android retention risks at corpus scale.



Introducing LEAKSCOPE

A SOOT based static analysis framework that targets lifecycle aware leak prone retention patterns in compiled Android applications at corpus scale.





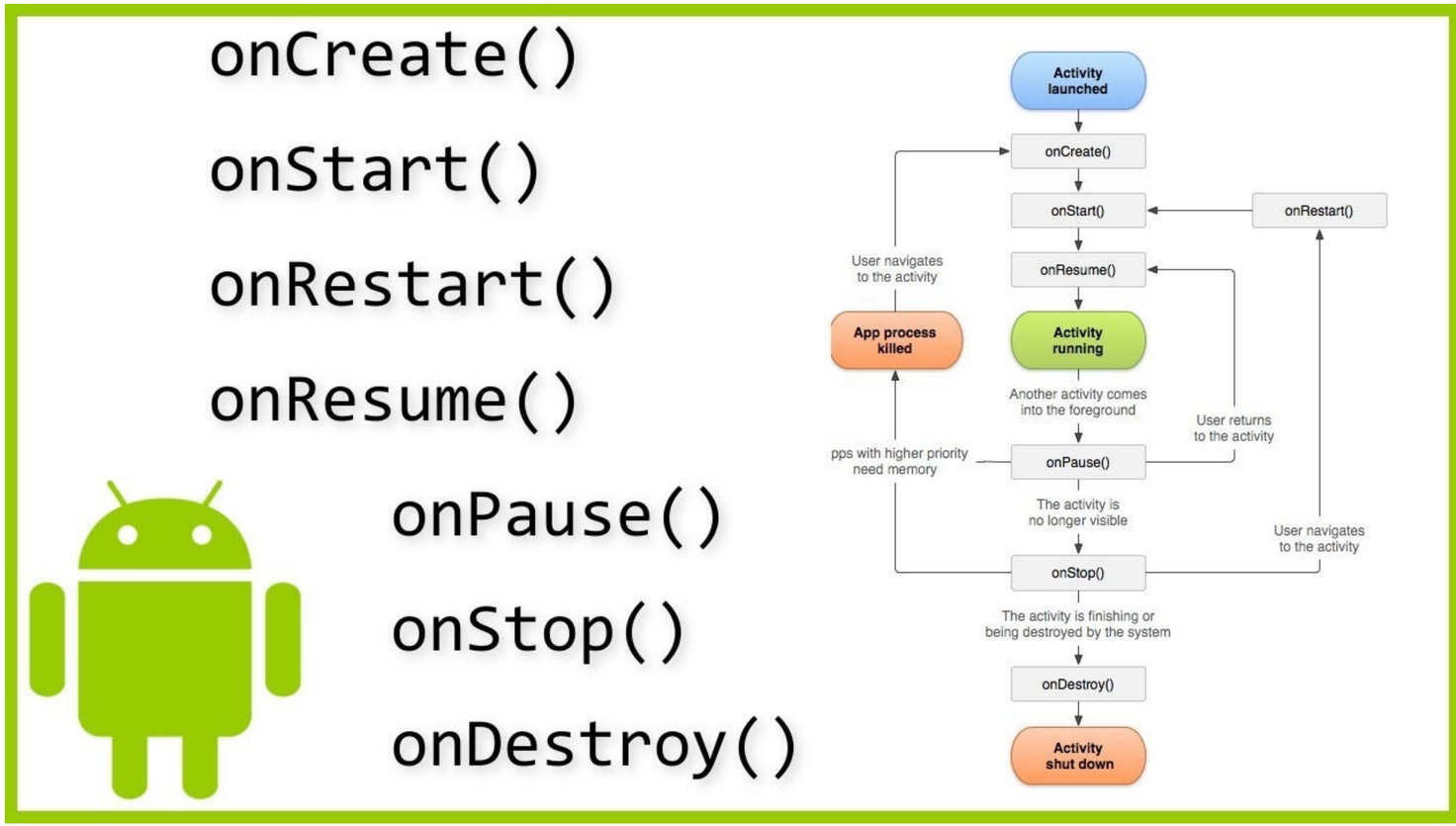
The Pipeline

DataSet

1. JLeaks
2. Defects4J
3. Bugs.jar
4. Vul4J







Detector Taxonomy

LEAKSCOPE

Detector Family	Category	Static Rule
Threaded UI Reference	Core leak	Worker execution capturing activities, fragments, or views
Fragment View Field Retention	Core leak	View/binding fields not released in `onDestroyView()`
State Holder Leak	Core leak	Static/long-lived holders retaining lifecycle-sensitive references
ViewModel Context Reference	Core leak	ViewModel fields involving contexts, activities, or views
Flow Retention	Core leak	Flow collection not guarded by lifecycle-aware APIs
Improper Callback Registration	Core leak	Callbacks registered without LifecycleOwner binding
BillingClient Cleanup	Core leak	BillingClient without `endConnection()`
Service Resource Management	Adjacent	Started services lacking `stopSelf()` paths
View Binding Opportunity	Auxiliary	Repeated `findViewById()` use

Benchmark Corpus

627

Compiled Projects

Successfully built and analyzed

~10K

Repos Queried

From GitHub — steep attrition typical of build replication studies

533

Unique Owners

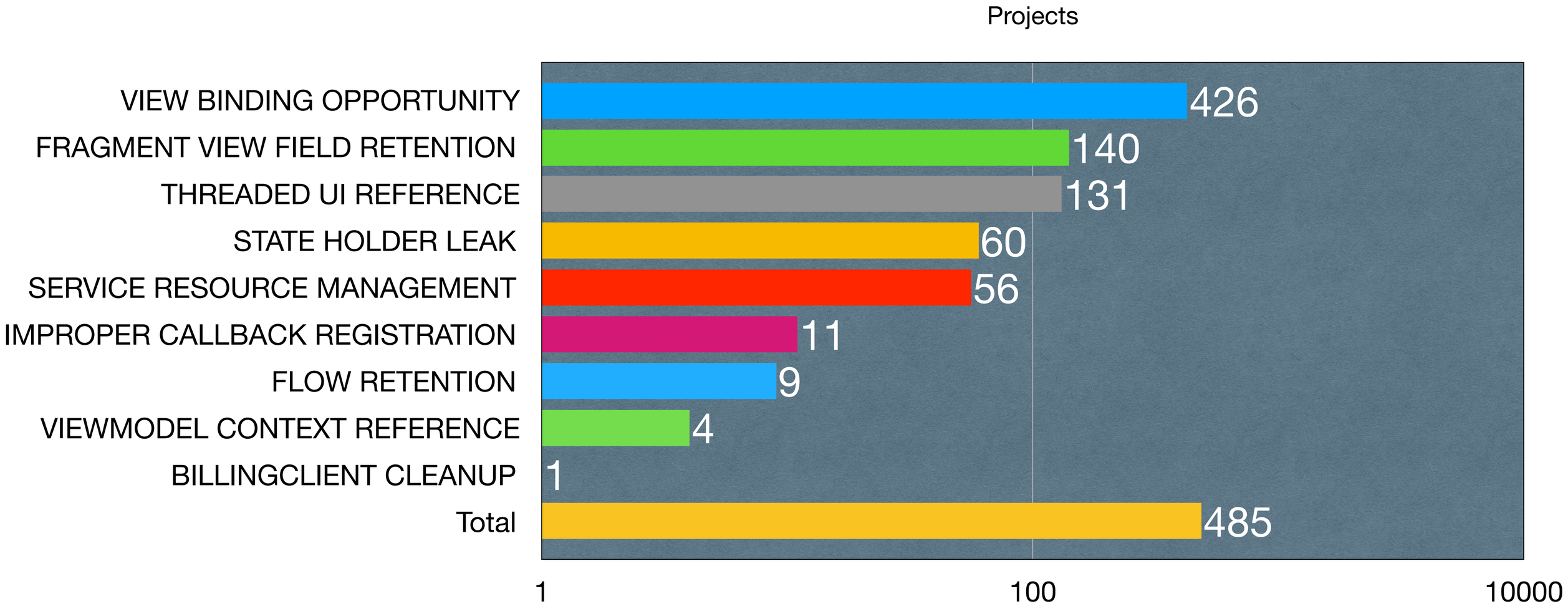
Diverse project provenance

Corpus Profile

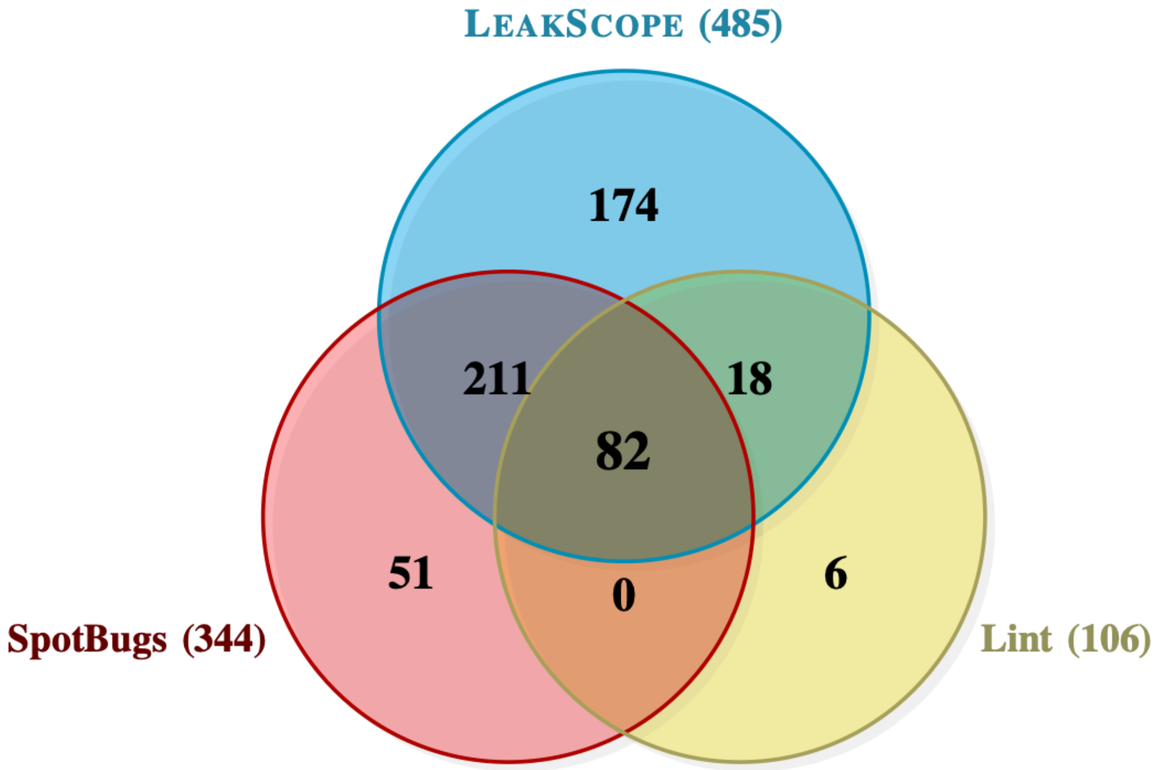
Property	Value
Primary language Java	594 (94.7%)
Primary language Kotlin	16 (2.6%)
Creation year range	2009–2026
Star count median / max	92 / 37,759
Repo size median	2.7 MB

ⓘ Attrition primarily from legacy dependencies, missing SDKs, incompatible Gradle constraints, or hardcoded local configurations.

RQ1: Prevalence of Lifecycle-Aware Retention Patterns

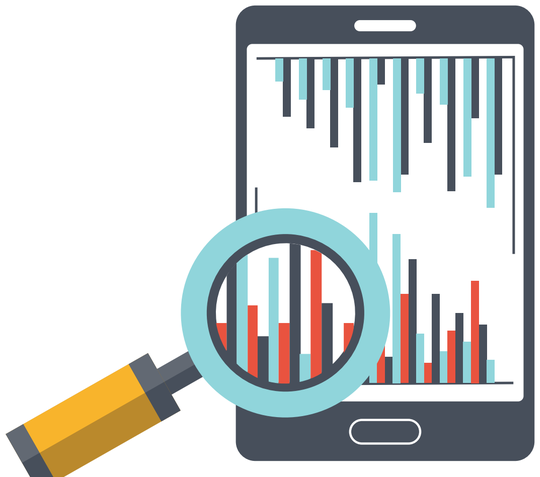


RQ2: Comparison Against Android Lint and SpotBugs



RQ2: Comparison Against Android Lint and SpotBugs

Category	LeakScope	Lint	SpotBugs	Lint ∩ LS	SB ∩ LS	Only LS
Fragment View Field Retention	140	0	0	0	0	140
Threaded UI Reference	131	16	93	5	42	84
State Holder Leak	60	70	292	17	57	3
Improper Callback Registration	11	0	0	0	0	11
Flow Retention	9	0	0	0	0	9
ViewModel Context Reference	4	0	0	0	0	4
BillingClient Cleanup	1	0	0	0	0	1
Core leak subtotal	356	86	385	22	99	252
Service Resource Management	56	0	0	0	0	56



RQ3: Precision and Manual Validation

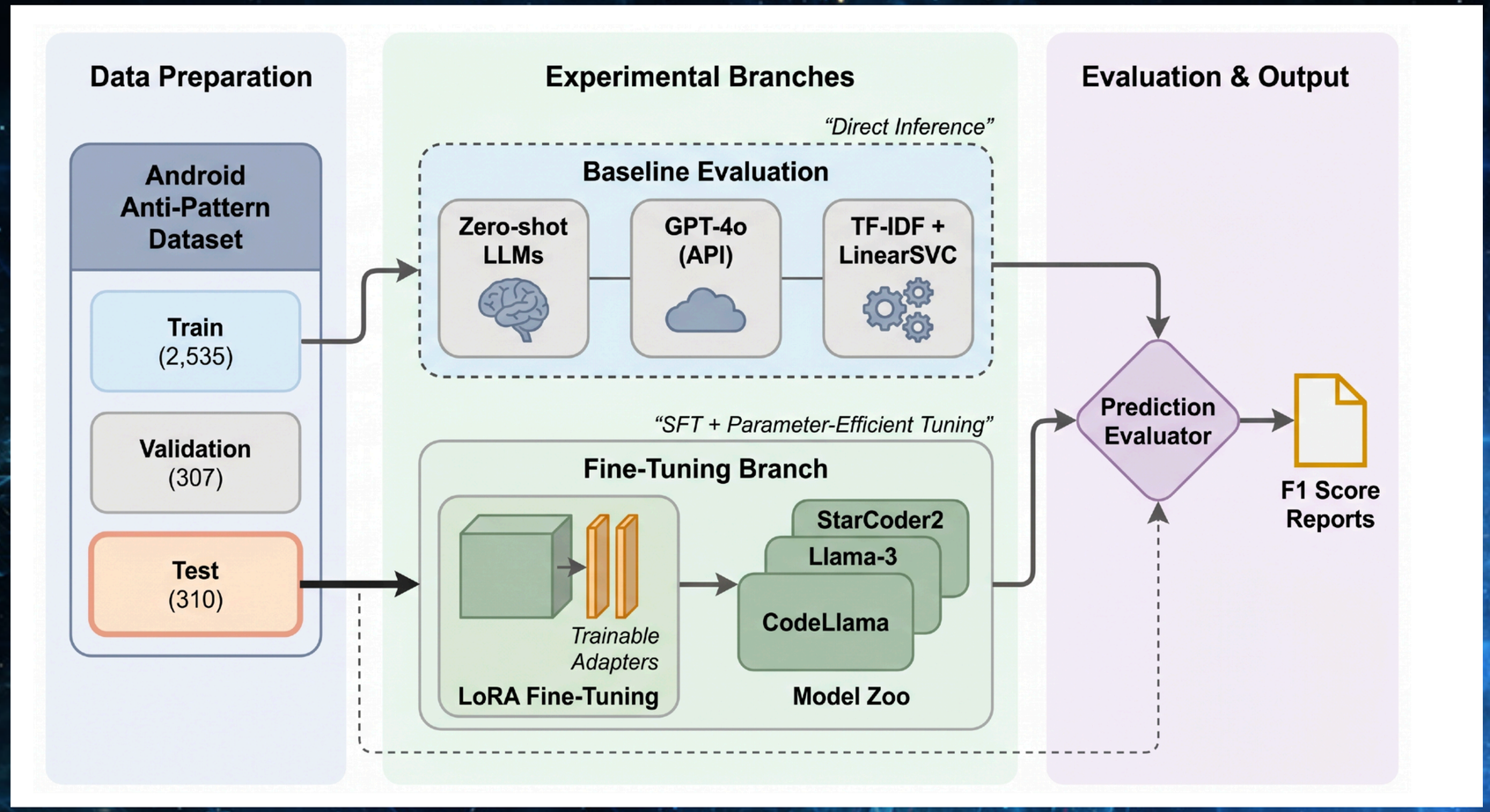
Detector	Sample	Population	Rate
View Binding Opportunity	100	1,688	5.9%
Fragment View Field Retention	80	684	11.7%
Threaded UI Reference	80	355	22.5%
State Holder Leak	60	106	56.6%
Service Resource Management	40	79	50.6%
Improper Callback Registration	14	14	100.0%
ViewModel Context Reference	9	9	100.0%
Flow Retention	4	12	33.3%
BillingClient Cleanup	1	1	100.0%
Total	388	—	—

RQ3: Precision and Manual Validation

Detector	TP	FP	UNS	Precision	κ
View Binding Opportunity	96	4	0	0.960	0.79
Fragment View Field Retention	80	0	0	1.000	1.00
Threaded UI Reference	75	4	1	0.949	0.59
State Holder Leak	40	18	2	0.690	0.85
Service Resource Management	37	3	0	0.925	0.84
Improper Callback Registration	14	0	0	1.000	1.00
ViewModel Context Reference	9	0	0	1.000	–
Flow Retention†	0	4	0	0.000	–
BillingClient Cleanup	1	0	0	1.000	–
Core leak subset	219	26	3	0.894	–
Overall suite	352	33	3	0.914	0.80



Future Direction: LLM Fine Tuning for Android Anti-Pattern Detection



EXPANSE

SAN DIEGO SUPERCOMPUTER CENTER

UNIVERSITY OF CALIFORNIA SAN DIEGO

Setup

Component	Value
Open-source backbones	CodeLlama-7B/13B, Llama-3-8B, StarCoder2-7B
Adaptation	LoRA (PEFT)
LoRA rank / α / dropout	8 / 16 / 0.05
LoRA target modules	$\{q_{\text{proj}}, v_{\text{proj}}\}$
Max sequence length	1,024 tokens (2,048 in ablation)
Epochs	3
Batch size (per device / total)	1 / 4
Optimizer / LR schedule	AdamW / cosine, peak 2×10^{-4}
Warmup ratio / weight decay	0.03 / 0.001
Precision	bfloat16
Distributed backend	DeepSpeed ZeRO-2 (CPU optimizer offload)
Hardware	4×A100 40 GB
Decoding	Greedy, max_new_tokens=300
Evaluation split	310 examples, 5 detectors

Motivation for Fine tuning

Detector: `ThreadedUIReference` **True Label:** `Yes`

Input: Android Java Snippet (Row 62)

```
1 toolbar.post(() -> {
2   new Thread(() -> {
3     Thread.sleep(500);
4     runOnUiThread(this::init  // <-- Capture of
      UI context inside worker thread
5 // ... [truncated for length]
```

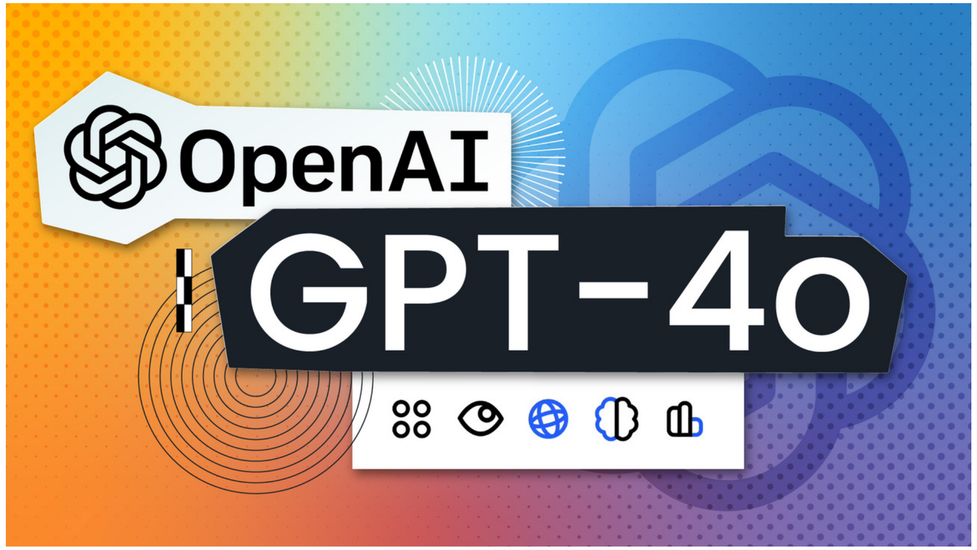
Model Predictions

GPT-4o (Zero-shot) ✗

"No, the provided code does not contain a ThreadedUIReference violation..."

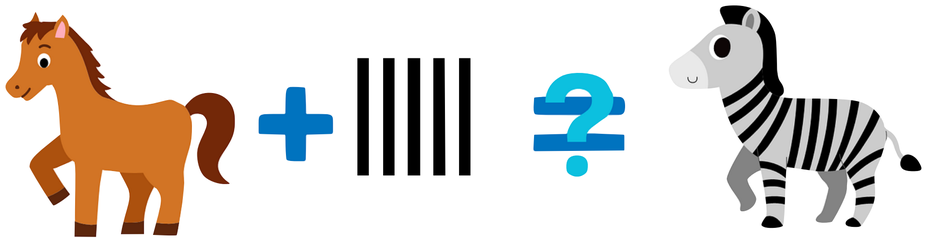
StarCoder2-seq2048 (Fine-tuned) ✓

"Yes. Explanation: Scenario 1: Worker thread holds UI object reference."

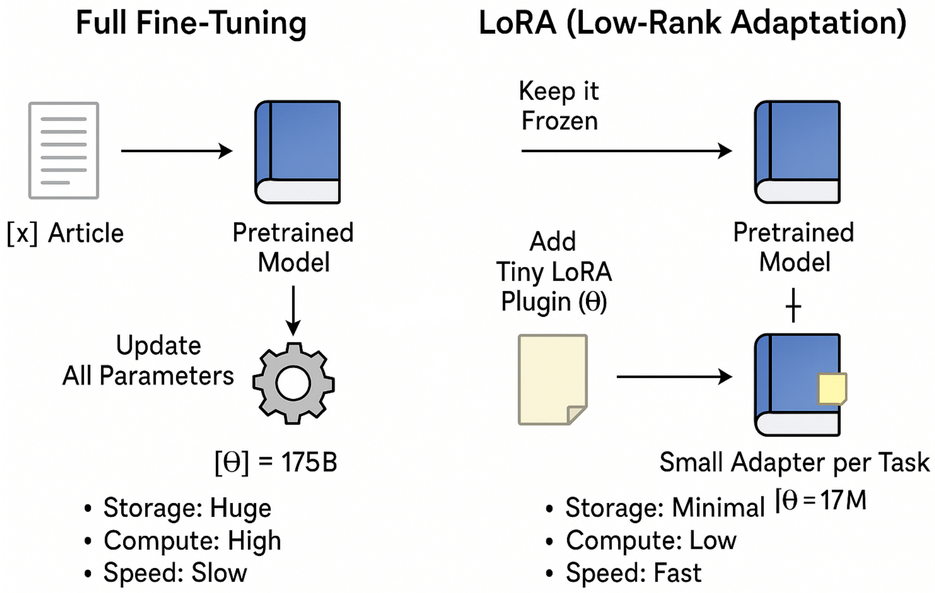




Results So Far!



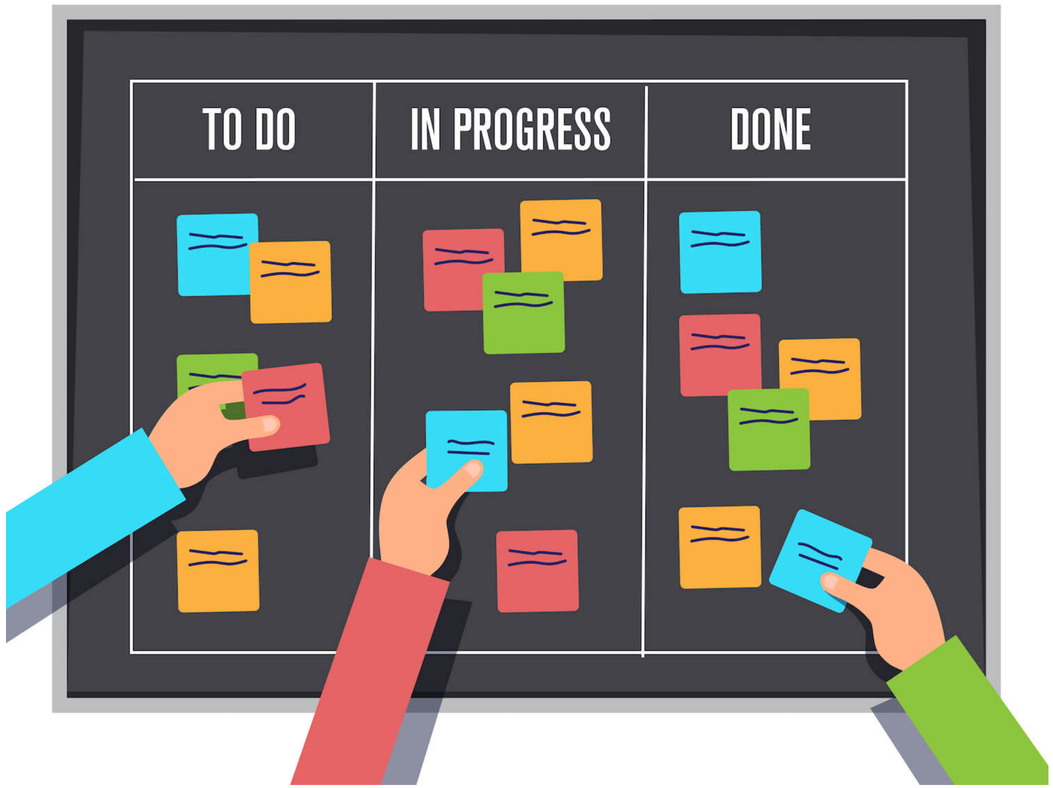
Zero-shot Learning



Model	Prec.	Rec.	F1	Acc.
TF-IDF + SVM	0.286	0.263	0.274	0.281
Zero-shot open-source				
CodeLlama-7B (ZS)	0.000	0.000	0.000	0.484
CodeLlama-13B (ZS)	0.429	0.019	0.036	0.481
Llama-3-8B (ZS)	0.558	0.181	0.274	0.503
StarCoder2-7B (ZS)	0.000	0.000	0.000	0.484
Fine-tuned open-source (LoRA, default)				
CodeLlama-7B (FT)	0.586	0.106	0.180	0.500
CodeLlama-13B (FT)	0.477	0.131	0.206	0.477
Llama-3-8B (FT)	0.438	0.087	0.146	0.471
StarCoder2-7B (FT)	0.529	0.625	0.573	0.519
StarCoder2-7B ablations (Sec. VII-E)				
+ seq 2,048 (q,v, r8)	0.526	0.762	**0.622**	0.523
+ r16, seq 2,048, q,v, 5ep	0.531	0.637	0.580	0.523
+ all-7 modules (seq 1,024)	0.442	0.237	0.309	0.452
+ r16, seq 2,048, all-7	0.663	0.419	0.513	0.590
Proprietary reference				
GPT-4o (ZS)	0.792	0.644	*0.710*	*0.729*

Ongoing Tasks

- Reporting Findings to Developers
- Measuring Tool Adoption and Impact
- Results Analysis
- Agentic framework integrated to LLMs



Plan

1. **Spring 2026**

   Submit Paper 1 to **ICSE 2027**

2. **Summer 2027**

   Submit LLM work to **ISSRE 2026 Doctoral Symposium**

3. **2026**

   Waiting for Commets from ACM Computing Surveys:
   Side Channel Attacks in Software Defined Networks

4. **2026 Defense**

   Dissertation writing and defense



Thank You!

